

Linux for Bioinformatics - Comprehensive Lecture Notes

Instructor: Uche Stephen.

Date: April 6 2025.

Table of contents:

1. Introduction to Linux in Bioinformatics
2. Setting Up Ubuntu with Windows Subsystem for Linux (WSL)
3. Understanding the Linux Kernel and Directory System
4. Linux Shells, Terminals, and Command-Line Basics
5. Essential Linux Commands for Bioinformatics Workflows
6. Files, Ownership, and Permissions
7. Sorting, Pattern Matching, and Text Processing (sort, uniq, grep, sed, awk)
8. Installing Bioinformatics Software
9. Environment Variables, Bash Variables & Bash Arrays
10. Bash Scripting & Automation for Bioinformatics

1. Introduction to Linux in Bioinformatics

Linux is the backbone of most bioinformatics workflows. It is a powerful, open-source operating system that enables scientists to automate repetitive tasks, run high-throughput data analysis, and leverage computational tools efficiently.

Linux is the **most widely used operating system** in scientific computing and bioinformatics. From genome sequencing to protein modeling, most tools and workflows are developed and optimized to run on Linux.

In bioinformatics, the majority of software tools (e.g., samtools, bedtools, GATK) are designed for Linux environments due to their flexibility, security, community support, and frequent updates.

What is Linux?

Linux is a **Unix-like, open-source, multi-user, and multitasking** operating system. It serves as the core of servers, mobile devices, embedded systems, supercomputers, and most importantly for us—**computational biology research environments**.

- **Multi-user:** Multiple users can access and work on the same machine without interfering with each other.
 - **Multi-tasking:** It can handle many tasks at once—perfect for long-running sequence alignment or ML jobs.
-

Why Bioinformaticians Prefer Linux

Bioinformatics workflows often involve **large text-based files** (like FASTA, VCF, GTF, and FASTQ) that need to be manipulated, filtered, and analyzed. Linux provides powerful command-line tools and scripting capabilities that make this process efficient and reproducible.

Feature	Benefit in Bioinformatics
Command-Line Tools	Ideal for processing large sequence files with simple commands

Scriptability	Automate repetitive tasks (e.g., quality control, filtering, annotation)
Reproducibility	Shell scripts ensure workflows can be exactly replicated
Open-source Ecosystem	Many tools (e.g., samtools, bedtools, bwa) are built for Linux
Stability	Suitable for long-running, memory-intensive genomic analyses

Real Use Cases in Bioinformatics

Task	Linux Tools Commonly Used
RNA-seq quality control	FastQC, multiqc, trimmomatic
Genome alignment	bwa, bowtie2
Variant calling	samtools, bcftools, GATK
CNV analysis	ASCAT, CNVkit, bedtools
RNA expression quantification	htseq, featureCounts
Protein interaction mapping	grep, awk, BLAST
ML-based prediction models	Python scripts running via Linux CLI

2. Setting Up Ubuntu with Windows Subsystem for Linux (WSL)

What is WSL?

Windows Subsystem for Linux (WSL) allows users to run a full Linux distribution—like Ubuntu—on a Windows machine **without needing a virtual machine or dual-boot setup**. It's ideal for users who prefer working in Windows but need the power and flexibility of Linux.

WSL is particularly useful for:

- Running Linux-based bioinformatics tools
 - Accessing the Bash terminal
 - Scripting and automating analysis tasks
 - Reproducibility and compatibility with Linux servers
-



Step-by-Step Guide to Installing Ubuntu via WSL



Method 1: Using Windows GUI

1. Enable WSL Feature

- Open the Start Menu and search: **Turn Windows features on or off**
- Check  the box for **“Windows Subsystem for Linux”**
- Also check  **“Virtual Machine Platform”**
- Click **OK** and **Restart** your computer.

2. Install Ubuntu

- Open the **Microsoft Store**
- Search for **Ubuntu 22.04.3 LTS, or any other version of your choice**
- Click **Install**
- Once installed, launch the app and follow the prompts to set your **username** and **password**. Note that when you are inputting your password, it will not appear on the screen, but it is being recorded by your computer.

3. Set WSL2 as Default Version

In your Windows start menu, search and open **PowerShell as Administrator** and run:

```
wsl --set-default-version 2
```

If prompted, install the kernel update from:

 <https://aka.ms/wsl2kernel>

Most errors that are usually encountered during WSL installation become solved just by installing this kernel package.

4. Verify Installation

```
wsl -l -v
```

This lists all installed distros (i.e. Operating systems built upon the Linux kernel and are usually called Linux distributions) and confirms that Ubuntu is running on version 2.

✅ Method 2: Use PowerShell (Command-Line Only)

Run the following command in PowerShell **as Administrator**:

```
Enable-WindowsOptionalFeature -Online -FeatureName  
Microsoft-Windows-Subsystem-Linux
```

Restart your system. Then, install Ubuntu as described above in Method 1.

First Time Setup

After launching Ubuntu:

- You'll be asked to create a **username** and **password**

- This is your Linux login for the terminal and acts like a local Linux environment on your Windows machine. Note again that when you are inputting your password, it will not appear on the screen, but it is being recorded by your computer.
-

Verifying Ubuntu and Terminal Functionality

Open your Ubuntu terminal and try running:

```
whoami                # Should return your Linux username
uname -a              # Shows Linux system info
```

Pro Tip:

You can also access your Linux files from Windows using:

```
\\wsl.localhost\Ubuntu-22.04\
```

And access Windows files from Linux via:

```
cd /mnt/c/Users/YourUsername/
```

3. Understanding the Linux Kernel and Directory System

What is the Linux Kernel?

Linux is modular (i.e. it contains kernel modules), composed of a kernel and various utilities. The **Linux Kernel** is the **core component/engine** of the Linux operating system. It sits between the hardware and user applications, managing:

- **Memory**

- **Processes**
- **Hardware devices**
- **System calls**
- **Users and permissions**

It is responsible for all interactions between the software and hardware. When you run a command, open a file, or launch a program, the kernel ensures those actions are executed efficiently and securely.

Linux is Built Around Files

In Linux, **everything is a file** — this includes:

- Text files
 - Directories
 - Devices (e.g., `/dev/sda1`)
 - Processes
 - System settings (e.g., `/proc/cpuinfo`)
-

The Linux Directory Structure

Linux follows a **hierarchical file system** that starts from the root (`/`) directory. Here's a breakdown of key system folders:

Directory	Purpose
<code>/</code>	The root directory – everything starts here
<code>/home</code>	User directories (e.g., <code>/home/alice</code>)

<code>/bin</code>	Essential system binaries (commands like <code>ls</code> , <code>cp</code>)
<code>/etc</code>	System-wide configuration files
<code>/usr</code>	User-installed applications and libraries
<code>/var</code>	Variable files – logs, mail, print queues
<code>/tmp</code>	Temporary files
<code>/dev</code>	Device files (hard drives, USBs)
<code>/proc</code>	Kernel and process information (virtual filesystem)

Running:

```
ls / # Will List all root directory contents
```

Your Home Directory

Every user in Linux has a **home directory**:

- For example: `/home/heizenberg_adeyemi`
- It's the default starting point when you open the terminal

You can return there anytime using:

```
cd ~
```

```
# or simply
```

```
cd
```

Viewing the Directory Structure

Try this to view the structure and understand its layout:

```
ls /
```

```
ls /home
```

```
ls /etc
```

To visualize your current location, run:

```
pwd
```

The output of this command, for example, will appear like:

```
/home/heizenberg_adeyemi/docu/bioinformatics
```

4. Linux Shells, Terminals, and Command-Line Basics

What is a Shell?

A **shell** is a command-line interpreter that allows users to **interact with the operating system**. It interprets text commands and passes them to the Linux kernel.

Popular Linux shells include:

Shell	Description
Bash (Bourne Again Shell)	Most common; used in Ubuntu and most Linux distros
Zsh	A more feature-rich shell with autocompletion and plugins
Sh	The original Bourne shell
Tcsh	Based on the C shell; supports scripting in C-like syntax

What is the Terminal?

The **Terminal** is the visual interface for interacting with your shell, i.e., it is what you see on the screen where you type your commands.

Common Shell Features

- **Prompt:** The command-line interface (e.g., `heizenberg_adeyemi@LAPTOP-HTI5NM9J:~$`)
 - **History:** Use the arrow keys `↑/↓` to navigate previous commands
 - **Tab completion:** Type part of a command or filename, press `Tab` to auto-complete
 - **Command chaining:** Use `&&`, `;`, `|` to combine commands
-

Why Learn the Shell?

Advantage	Use Case
Speed	Execute commands faster than clicking through menus
Automation	Write scripts to process data or manage files
Precision	Combine multiple commands using pipes
Reproducibility	Record exact steps in a script for future use or publication

Examples of Common CLI Usage

1. **Check current directory**

`pwd`

Output: `/home/heizenberg_adeyemi/docu`

2. **List contents**

`ls -l`

3. **Change directories**

`cd ~/myfolder`

4. **Make a new folder**

`mkdir project1`

5. **Move into it**

`cd project1`

6. **Create a file**

`touch data.txt`

7. **View the file**

`cat data.txt`

Command-Line vs GUI (Graphical Interface)

Action	CLI	GUI
Create a folder	<code>mkdir results</code>	Right-click → New Folder
Move a file	<code>mv file.txt folder/</code>	Drag and drop
Search contents	<code>grep "gene" data.txt</code>	Ctrl + F (only inside open file)

Useful Keyboard Shortcuts in the Terminal

Shortcut	Function
----------	----------

<code>Ctrl + A</code>	Move to beginning of line
<code>Ctrl + E</code>	Move to end of line
<code>Ctrl + U</code>	Delete from cursor to beginning
<code>Ctrl + K</code>	Delete from cursor to end
<code>Ctrl + L</code>	Clear screen (like <code>clear</code>)
<code>Ctrl + R</code>	Search command history

🧩 Shell vs Bash vs Terminal vs Linux

Term	Meaning
Shell	Interpreter that processes commands
Bash	A specific shell (Bourne Again Shell)
Terminal	The interface where the shell runs
Linux	The entire operating system

Understanding Absolute vs. Relative Paths

When navigating or referencing files and directories in Linux, you'll often use two types of paths:

♦ Absolute Path

- Starts from the **root** directory `/` and points to the exact location of a file or folder.
- Works **regardless** of your current location in the file system.

Example:

```
cd /home/heizenberg_adeyemi/projects/cancer_data
```

This navigates **directly** to the specified folder from the root.

♦ **Relative Path**

- It starts from your **current location** in the directory structure.
- It uses symbols like:
 - `.` = current directory
 - `..` = parent directory

Example:

```
cd ../results
```

If you're in `/home/heizenberg_adeyemi/projects/scripts`, this command will take you to `/home/heizenberg_adeyemi/projects/results`.

📌 **Practical Comparison:**

Path Type	Command	Description
Absolute	<code>cd /usr/local/bin</code>	Goes to that exact location from root
Relative	<code>cd ../../coding</code>	Moves up two directories, then enters <code>coding</code>

Using the **right path type** improves scripting, organization, and portability in collaborative bioinformatics workflows.

In Linux, **options are flags or switches used with commands to modify their behavior**. They provide different ways to interact with the command line. Common options include `-h` or `--help` for help, `-v` or `--version` for version information, and `-l` for `ls -l`.

5. Essential Linux Commands for Bioinformatics Workflows

This section introduces **fundamental Linux commands** you'll frequently use in a bioinformatics environment. These tools help navigate filesystems, manage datasets, automate tasks, and process large files—all through the **command-line interface (CLI)**.

Directory and File Navigation Commands

Command	Description	Example
<code>pwd</code>	Print current working directory	<code>pwd</code>
<code>ls</code>	List contents of directory	<code>ls -l</code>
<code>cd</code>	Change directory	<code>cd /home/heizenberg_adeyemi/coding/</code>
<code>mkdir</code>	Make a new directory	<code>mkdir new_data</code>
<code>rmdir</code>	Remove an empty directory	<code>rmdir new_data</code>

File Creation, Moving, and Deletion

Command	Description	Example
---------	-------------	---------

<code>touch</code>	Create a new empty file	<code>touch results.txt</code>
<code>cp</code>	Copy the contents of a file or directory	<code>cp genome.fasta genome_2.fasta</code>
<code>mv</code>	Move or rename files	<code>mv old_data.txt new_data.txt</code>
<code>rm</code>	Remove a file	<code>rm results.txt</code>

File Types and Text Editors in Linux

- **Shell Script Files (.sh):** .sh files contain sequences of commands for automation, written in languages like Bash or sh.
- **Comparing Text Editors:**
 - **Nano:** A simple, beginner-friendly editor with single-mode operation and a menu of commands at the bottom.
 - **Vim:** Advanced, with multi-mode operation (Insert, Normal, Command), suitable for users needing powerful text manipulation.

Usage: To create or edit a file, type `vim filename.txt`.

- **Insert Mode:** Press `i` to start writing.
 - **Command Mode:** Press `Esc` to exit Insert Mode.
 - **Save:** Type `:w` to write changes.
 - **Quit:** Type `:q` to exit without saving or `:wq` to save and quit.
 - **Force Quit:** Use `:q!` to quit without saving changes.
 - **Modes in Vim:** Vim has two main modes:
 - **Insert Mode (i):** Insert text before the cursor.
 - **Append Mode (a):** Insert text after the cursor.
 - **File Recovery in Vim:** If needed, `vim -r filename.txt` can be used to recover an unsaved file.
-

Viewing File Contents

Command	Description	Example
<code>cat</code>	Display contents of file	<code>cat filename.txt</code>

<code>less</code>	View long files one screen at a time	<code>less gene_list.txt</code>
<code>head</code>	Show the first 10 lines	<code>head genome.fasta</code>
<code>tail</code>	Show the last 10 lines	<code>tail -n 20 genome.fasta</code>

File Analysis Tools

1. Word Count

<code>wc -l genome.fasta</code>	<code># Count lines</code>
<code>wc -w gene_list.txt</code>	<code># Count words</code>
<code>wc -c filename.txt</code>	<code># Count characters</code>

2. Sorting and Uniqueness

<code>sort gene_list.txt</code>	<code># Alphabetically sort lines</code>
<code>uniq sorted_genes.txt</code> (requires sorted input)	<code># Remove duplicate lines</code>
<code>sort genes.txt uniq > clean.txt</code>	<code># Combined</code>

3. Search with Grep

<code>grep "Chromosome" gene_list.txt</code>	<code># Find lines with "Chromosome"</code>
<code>grep -i "Chromosome" gene_list.txt</code>	<code># Case-insensitive search</code>

4. Redirection and Piping

Redirection and piping are essential tools in Linux that allow you to **chain commands**, **store outputs**, and **manipulate data efficiently**—particularly useful when handling large bioinformatics datasets like FASTA, VCF, or gene expression matrices.

Redirection

Redirection sends the output of a command to a file **instead of displaying it** on the screen.

Symbol	Function	Description	Example
>	Output Redirection	Overwrites the target file with the command output.	<code>ls > file_list.txt</code>
>>	Output Append	Appends the output to the target file without overwriting.	<code>echo "BRCA1" >> cancer_genes.txt</code>
<	Input Redirection	Uses a file as input to a command.	<code>wc -l < cancer_genes.txt</code>

The `echo` command in Linux is a built-in command that allows users to display lines of text or strings that are passed as arguments. It is commonly used in shell scripts and batch files to output status text to the screen or a file.

Piping (|)

Piping **connects the output of one command as the input to another**, creating a streamlined **processing pipeline**.

Example 1: Find FASTA files and count them

```
ls *.fasta | wc -l
```

Finds all `.fasta` files in the directory and counts them.

Pro Tips

- Use `man <command>` to read manuals for commands: e.g `man grep`
 - Use **Tab** for autocomplete and `↑ / ↓` arrows to browse command history.
 - Use `clear` to tidy the terminal view.
-

6. Files, Ownership, and Permissions

File Ownership and Permissions in Linux

Linux uses a **user-group-other** model for setting access controls on files and directories.

File Permission Syntax:

```
-rwxr-xr-- 1 faruq bioinfo 2352 Jul 2 08:22 data.txt
```

- **rwx** → Owner can read/write/execute
 - **r-x** → Group can read/execute
 - **r--** → Others can only read
 - **faruq** → Owner
 - **bioinfo** → Group
-

Change Permissions with **chmod**

Using numeric values:

```
chmod 755 script.sh # Owner: rwx, Group: rx, Others: rx
```

Value	Permission
7	read + write + execute
6	read + write
5	read + execute
4	read only

0 no permission

Using symbolic mode:

```
chmod u+x file.txt      # Add execute to user  
chmod g-w file.txt      # Remove write from group
```

Changing File Ownership: `chown` and `chgrp`

```
sudo chown alice:research file.txt
```

- Changes file owner to `alice` and group to `research`

```
sudo chgrp bio_team report.csv
```

- Changes only the group

To check your groups, run:

```
groups
```

N.B: `chmod` changes the permissions of a file, `chown` changes user ownership of a file, and `chgrp` changes group ownership of a file.

7. Sorting, Pattern Matching, and Text Processing (sort, uniq, grep, sed, awk)

In computational biology, massive text-based datasets such as **FASTA**, **GFF**, **VCF**, and **expression matrices** are the norm. Linux provides powerful utilities to **sort**, **filter**, **search**, and **transform** these files directly from the terminal.

This section introduces 5 indispensable tools for bioinformatics text processing: `sort`, `uniq`, `grep`, `sed`, and `awk`.

1. Sorting with **sort**

Used to sort the lines in a file numerically or alphabetically.

```
sort genes.txt
sort -r genes.txt      # Reverse order
sort -n expression.txt # Sort numerically
```

 **Bioinformatics Use Case:** Sort differentially expressed genes by p-value.

2. Removing Duplicates with **uniq**

uniq eliminates **adjacent** duplicate lines. Typically used with **sort**. e.g:

```
sort gene_ids.txt | uniq
uniq -c sorted_genes.txt # Count duplicate lines
```

 **Tip:** Always **sort** the file before using **uniq**.

3. Pattern Matching with **grep**

Search for **specific patterns or sequences** in files.

```
grep "BRCA1" genes.fasta      # Search exact matches
grep -i "tp53" mutations.txt  # Case-insensitive
grep -r "oncogene" ./results/ # Recursive search in
directory
grep -v "normal" annotations.txt # Invert match (exclude
pattern)
```

4. Text Replacement with **sed**

`sed` (stream editor) edits files line-by-line. Most commonly used for find-and-replace.

```
sed 's/mutant/wildtype/' samples.txt    # Replace first instance
per line
```

```
sed 's/mutant/wildtype/g' samples.txt    # Replace all
instances per line
```

```
sed -i 's/chr/CHR/g' bed_file.bed        # In-place edit of
file
```

⚠ Use `-i` with caution – it overwrites files.

5. Text Analysis and Column Extraction with `awk`

`awk` is ideal for analyzing **column-based** data.

Basic Structure:

```
awk '{print $1, $3}' genes.tsv
```

Specify delimiter (e.g., for CSV):

```
awk -F "," '{print $2}' data.csv
```

Condition-based filtering:

```
awk '$3 > 0.5' expression_data.txt      # Show lines where 3rd
column > 0.5
```

Add header row logic:

```
awk 'NR==1{print "Gene\tLogFC"} NR>1{print $1"\t"$3}' results.txt
```

Use Case: Find genes mutated in cancer samples

```
grep "TP53" mutations.vcf | awk '{print $1, $2, $4, $5}'
```

Combining Tools

```
cat variants.txt | grep "missense" | sort | uniq -c
```

This filters for missense mutations, sorts them, and counts how many times each appears.

8. Installing Bioinformatics Software

Running bioinformatics workflows often requires installing specialized tools like **FastQC**, **BWA**, **GATK**, or **Samtools**. In Linux, software can be installed using **package managers**, **precompiled binaries**, **source code**, or **environment managers** like **conda**.

1. Using **apt** (Advanced Package Tool)

apt is the default package manager for Debian-based systems like Ubuntu.

Syntax:

```
sudo apt update          # Update the package list
sudo apt install tool    # Install a tool
```

Example:

```
sudo apt install fastqc
```

The **sudo** command in Linux (and other Unix-like operating systems) **allows users to execute commands with elevated privileges, typically the superuser or root privileges**. It's a crucial tool for system administration and other tasks that require access to restricted resources.

Note: Not all bioinformatics tools are available via **apt**. Some are hosted on other platforms or repositories.

2. Installing Samtools (Example from Ubuntu/Linux)

```
sudo apt update
sudo apt install samtools
```

To verify installation:

```
samtools --version
```

3. Using **conda** (Recommended for Bioinformatics)

Conda is an environment manager that allows isolated installations of software packages.

Install Miniconda (lightweight version):

```
wget  
https://repo.anaconda.com/miniconda/Miniconda3-latest-Linux-x86_64.sh
```

```
bash Miniconda3-latest-Linux-x86_64.sh
```

Restart terminal and add conda to **PATH** if needed.

Using Conda to Install Tools

```
conda create -n bio_env fastqc bwa samtools
```

```
conda activate bio_env
```

Install packages individually:

```
conda install -c bioconda bowtie2
```

```
conda install -c bioconda trimmomatic
```

The **-c bioconda** flag tells conda to use the **BioConda channel**, which specializes in life science software.

4. Install via Source Code

For tools not available via package managers or conda.

Example: Installing BEDTools from source

```
git clone https://github.com/arq5x/bedtools2.git
```

```
cd bedtools2
```

```
make
```

5. Using Precompiled Binaries

Some tools come with ready-to-use binaries.

Example:

```
wget https://github.com/xyz/tool/releases/tool.tar.gz
```

```
tar -xvzf tool.tar.gz
```

```
cd tool
```

```
./tool --help
```

6. Installing Software in WSL (Windows Subsystem for Linux)

WSL supports `apt`, `conda`, `wget`, and everything above. Just install tools as you would in Ubuntu.

Example:

```
sudo apt install fastqc
```

Check using:

```
which fastqc
```

Update & Manage Software

- `sudo apt upgrade` – Update all packages
 - `conda update fastqc` – Update individual conda package
 - `conda env list` – Show installed environments
-

Tool	Purpose
FastQC	Quality control of FASTQ files
BWA	Sequence alignment
Samtools	BAM file manipulation
GATK	Variant calling
Bedtools	Genomic interval operations

Use Case: Running FastQC on a CNV FASTQ File

```
mkdir fastqc_results  
fastqc sample1.fastq -o fastqc_results/
```

If **fastqc** is not found, install with:

```
sudo apt update  
sudo apt install fastqc
```

9. Environment Variables, Bash Variables & Bash Arrays

In Linux, variables are used to **store and manipulate values**, control how programs run, and pass information between shell sessions and processes. This section explains the differences between environment and shell variables, and how to use Bash arrays for bioinformatics workflows.

1. Environment Variables

Environment variables are **global settings** that affect how processes behave. They are inherited by child processes.

Viewing Environment Variables

```
printenv          # Lists all environment variables
```

```
printenv PATH     # Shows value of PATH variable
```

Setting Environment Variables

```
export GENOME="hg38"
```

```
echo $GENOME
```

 **export** makes a variable available to child processes.

Common Environment Variables

Variable	Description
<code>\$HOME</code>	User's home directory
<code>\$PATH</code>	Directories searched for executables
<code>\$USER</code>	Current logged-in user
<code>\$SHELL</code>	Current shell program
<code>\$PWD</code>	Present working directory

2. Shell (Bash) Variables

Shell variables are **local** and only apply to the current session or script.

Defining and Accessing

```
SAMPLE="BRCA1 "
```

```
echo $SAMPLE
```

3. Difference Between Shell and Environment Variables

Feature	Shell Variable	Environment Variable
Scope	Local to session	Inherited by child processes
Exportable	No	Yes (with <code>export</code>)
Persistence	Temporary	Can be stored in <code>.bashrc</code>

4. Bash Arrays

Arrays hold **multiple values** in one variable — useful for storing sample lists or chromosomes.

Defining Arrays

```
samples=("sample1" "sample2" "sample3")
```

Accessing Elements

```
echo ${samples[0]}          # First item
```

Looping Through Arrays

```
for s in "${samples[@]}; do
    echo "Running analysis on $s"
done
```

Bioinformatics Example

```
files=("BRCA.fastq" "TP53.fastq" "MYC.fastq")
```

```
for file in "${files[@]}; do
```

```
fastqc "$file" -o qc_output/  
done
```

This batch-processes FASTQ files with **FastQC** using a Bash array.

Persisting Variables

To make environment variables permanent, add them to `.bashrc`:

```
echo 'export TOOL_PATH="/usr/local/tools"' >> ~/.bashrc  
source ~/.bashrc
```

Practice Tasks

1. Define an array of chromosomes (`chr1`, `chr2`, `chrX`) and print them in a loop.
 2. Set and export a `DATA_PATH` variable for your datasets.
 3. Use `printenv` to list and confirm the variable is available.
-

10. Bash Scripting & Automation for Bioinformatics

1. What is a Bash Script?

A **Bash script** is a plain text file containing a series of commands that the Bash shell can interpret and execute.

Example script:

```
#!/bin/bash  
  
echo "Starting quality check..."
```

```
fastqc sample1.fastq -o qc_results/
```

Save it as:

```
nano run_fastqc.sh
```

Then make it executable:

```
chmod u+x run_fastqc.sh
```

Run the script:

```
./run_fastqc.sh
```

2. The Shebang (#!)

The first line of every script should be the **shebang**, indicating which interpreter to use:

```
#!/bin/bash
```

Other options:

- `/bin/sh`
 - `/usr/bin/env python3` (for Python)
 - `/usr/bin/env bash`
-

3. Bash Script Structure

```
#!/bin/bash
```

```
# Variables
```

```
SAMPLE="sample1.fastq"
```

```
# Commands
```

```
echo "Running QC on $SAMPLE"
```

```
fastqc "$SAMPLE" -o results/
```

4. Input Arguments (Positional Parameters)

Bash allows you to accept dynamic values via **positional arguments**:

```
#!/bin/bash

echo "Running QC on $1"

fastqc "$1" -o results/
```

Run it like:

```
./script.sh BRCA1.fastq
```

5. Sample Bioinformatics Script

Batch-process all `.fastq` files in a folder:

```
#!/bin/bash

mkdir -p qc_results

for file in *.fastq; do
    echo "Processing $file..."
    fastqc "$file" -o qc_results/
done
```

6. Conditional Logic and Loops

Conditional Execution

```
if [ -f "samples.txt" ]; then
    echo "File found!"
else
```

```
    echo "Missing file!"  
fi
```

7. Automating CNV Mapping (Example)

```
#!/bin/bash
```

```
ANNOTATION=annotation_genes.bed
```

```
mkdir -p mapped_output
```

```
for file in *_bed; do
```

```
    echo "Mapping $file..."
```

```
    bedtools intersect -a "$file" -b "$ANNOTATION" -wa -wb >  
    "mapped_output/${file%.bed}_mapped.bed"
```

```
done
```

8. Best Practices

- Always test your script on 1 or 2 samples first.
 - Use comments (`#`) to document steps.
 - Modularize scripts to simplify debugging.
 - Track versions of tools used.
-

Conclusion

Bash scripting forms the **automation backbone** of every robust bioinformatics project. Combined with environment variables, arrays, and looping constructs, you can easily create **reproducible pipelines** for genomics, transcriptomics, proteomics, or clinical data analysis.